

02

1,601,357

rows read.

50 rows returned.

One merchant. One query.

THE QUERY

A merchant opens their dashboard.

```
SELECT * FROM transactions
WHERE  merchant_id = ?
      AND  created_at > now() - interval '30 days'
ORDER BY created_at DESC
LIMIT 50;
```

You have written this endpoint. So have I. Several times.

THIS LOOKS FINE

```
CREATE TABLE transactions (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  merchant_id UUID          NOT NULL REFERENCES merchants (id),  
  reference    VARCHAR(255) NOT NULL,  
  amount       BIGINT       NOT NULL,  
  currency     VARCHAR(3)   NOT NULL,  
  status       VARCHAR(32)  NOT NULL,  
  provider     VARCHAR(64),  
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_transactions_merchant_id  
  ON transactions (merchant_id);
```

Typed types. Foreign key. An index on the column you filter by.
Nobody stops this in review.

HERE'S WHAT HAPPENS AT 4M ROWS

```
Parallel Bitmap Heap Scan on transactions
  Filter: (created_at > now() - '30 days')
  Rows Removed by Filter: 496159    (x3 workers)
Heap Blocks: exact=19255
Buffers: shared hit=1261 read=57143
→ Bitmap Index Scan on idx_transactions_merchant_id
   Index Cond: (merchant_id = '96444bf1...')
   rows=1601357

Execution Time: 227.885 ms
```

446 MB off disk. Two parallel workers. Three CPUs.
To return fifty rows.

SAME CODE. SAME INDEX.

	100,000 ROWS	4,000,000 ROWS
Rows read	40,026	1,601,357
Rows discarded	37,179	1,488,477
Rows returned	50	50
Read amplification	800 : 1	32,027 : 1
Read from disk	0 MB	446 MB
Execution time	6.2 ms	227.9 ms

The planner was never wrong. It estimated 2,820 rows and got 2,847.

The index is on merchant_id. The sort is on created_at. Nothing joins them.

ONE MONTH OF VOLUME

TABLE	ROWS	ON DISK
ledger_entries	7,199,500	794 MB
transactions	4,000,000	625 MB
transaction_attempts	4,599,493	469 MB

495 bytes per transaction, fully loaded — its ledger pair, its provider attempts, and every index on all three.

Year one at this rate: 190 million rows, 22 GB.

The biggest table is the one I never thought to size.

WHAT THE AVERAGE HIDES

MERCHANT	TRANSACTIONS	SHARE
96444bf1...	1,601,357	40.03%
c0fffc45...	107,459	2.69%
bc8b62f0...	44,429	1.11%
6d7bcabe...	34,323	0.86%

500 merchants. Average: 8,000 transactions each.

Capacity planned on that average is planning for a tenant who will never page you.

Capacity estimation isn't “will it be slow”. It's **when**, and at what row count.

Seed the real schema. Measure bytes per row. Project. Then check which tenant owns your table.

Laptop numbers: Postgres 15, 128 MB shared_buffers, cold cache. Your milliseconds will differ. The read ratio won't.

Post 03, Tuesday — idempotency keys.